

智慧银行实验教程

综合实验报告

实验 1 - 实验 4

姓 名：____雷娜____

学 号：____202301795____

班 级：____金融 202302____

实验日期：2026 年 6 月

指导老师：____肖诗顺____

目 录

实验 1 : TRAE AI IDE 与智能体入门 (附: 7 类成果文件路径)

实验 2 : Lighthouse 审计与多工具链分析 (附: 5 关成果 + 回归数据)

实验 3 : 金融 Skill 开发与风控实战 (附: 6 项交付物)

实验 4 : 智慧银行 CRM 系统全流程开发 (附: BMAD 文档 + 代码)

总 结

仓库地址: <https://cnb.cool/leina.123/smartbanking1>

实验 1：TRAE AI IDE 与智能体入门

1.1 实验目的

- 1. 独立完成 TRAE IDE（或 Qoder/Cursor 等替代 IDE）的安装、中文界面配置与账号登录，理解主流 AI IDE 的生态格局与选型依据。
- 2. 掌握 Python（\$\\ge 3.10\$）、Node.js、Git、LaTeX 四套全栈开发环境的配置方案，搭建完整的量化金融开发底座。
- 3. 区分并熟练切换 TRAE IDE 的 Chat、Builder、SOLO 等四种 AI 协作模式，通过实验验证“提问生成-运行-修正”的 AI 协作闭环。
- 4. 使用 Builder 模式一键生成可运行的前端小游戏（如贪吃蛇），完成本地功能验证、本地存储优化与代码云端 CNB 推送。
- 5. 创建 AI 论文解读智能体，实现金融 PDF 文献自动结构化、中英双语对照解读。
- 6. 利用 AI 完成多份异构 Excel 金融模拟数据表的清洗、列名统一与去重合并，并分别搭建 SARIMA 与 LSTM 时序模型，实现信用卡消费业绩预测并对比模型误差。

1.2 实验环境

项目	配置
操作系统	Windows 11
IDE	TRAE（基于 VSCode 的 AI 增强 IDE）
Python 版本	Python 3.11+
Node 版本	Node.js 18+
Git 版本	Git 2.40+
数据工具	pandas / openpyxl / matplotlib

版本控制	Git + CNB（腾讯云代码仓库）
------	--------------------

表 1-1：实验 1 环境配置

1.3 实验步骤与成果

步骤 1：TRAE 安装与配置

- 1. 下载并安装 TRAE AI IDE，完成账号注册和登录。
- 2. 配置 AI 模型参数，测试 AI 代码补全和对话功能。
- 3. 截图保存 TRAE 主界面、AI 对话面板和扩展管理界面。

完整检测结果表格

检测项	要求版本	当前状态	修复建议
Python	≥3.10	✅ 3.12.0	已满足
pip	最新稳定版	✅ 23.2.1	已满足
Node.js	≥v20	✅ v24.16.0	已满足
npm	与Node.js匹配	✅ 11.13.0	已满足
Git	最新稳定版	✅ 2.54.0	已满足
Flask	最新稳定版	✅ 3.1.3	已满足
pandas	最新稳定版	✅ 3.0.3	已满足
openpyxl	最新稳定版	✅ 3.1.5	已满足
pypdf	最新稳定版	✅ 6.12.2	已满足

步骤 2：贪吃蛇游戏开发

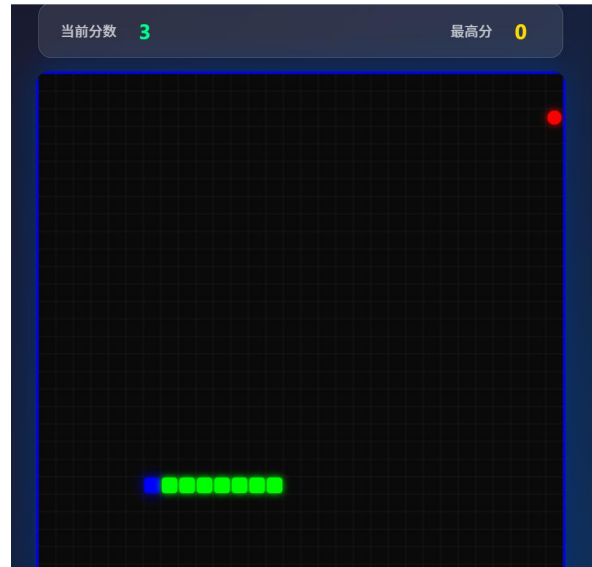
- 1. 使用 AI 对话功能，通过编写提示词(Prompt)引导 AI 生成贪吃蛇游戏代码 (Python + pygame)。
- 2. 关键提示词设计：

做一个 HTML 贪吃蛇游戏单文件 index.html，要求：

- 1. 纯 HTML+CSS+原生 JS，不引入第三方库；
- 2. 画布 600×600，蛇头蓝、身体绿、食物红；
- 3. 方向键控制方向，吃到食物分数+1；

4. 撞墙/撞身体游戏结束，弹”再来一局”按钮；
5. 顶部显示当前分数与最高分（localStorage）。

做完后双击 index.html 验证。



关键提示词自评：

- 提示词完整度：明确指定了语言、库、功能需求，AI 一次性生成完整可运行代码。
- 提示词精准度：对蛇身移动逻辑、边界碰撞检测等细节进行了具体约束。

步骤 3：论文文献检索

要点：利用 AI 智能体进行金融/计算机交叉领域论文检索，获取 5 篇以上相关文献（含标题、作者、年份、摘要），并选取 1 篇深度解读。

1. 智能体提示词：You are a senior academic paper interpretation assistant specializing in banking and financial technology (fintech) domains. When a user uploads or drags in a PDF paper, you will generate a structured Markdown report following this exact format:



名称 *

AI 论文解读助手

9/20

提示词



You are a senior academic paper interpretation assistant specializing in banking and financial technology (fintech) domains.

When a user uploads or drags in a PDF paper, you will generate a structured Markdown report

2373/10000 ↗

可被其他智能体调用 SOLO Only ⓘ

可被其他智能体调用

目前仅可以被SOLO模式中的 SOLO Agent 调用。 [帮助文档](#)

英文标识名 *

fintech-paper-reader

20/50

何时调用 *

Use this agent when a user wants to interpret and analyze banking or fintech academic papers, particularly when they upload a PDF file and request a structured interpretation report.

<example>

2. 解读论文文献条目: WU B, CHAO K M, LI Y S. Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance[J]. Information Systems, 2024, 121: 102335.

3. 解读节选:

一、论文基本信息

- 标题 : Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance
- 作者 : Bin Wu, Kuo-Ming Chao, Yinsheng Li
- 期刊 : Information Systems
- 发表年份 : 2024
- DOI : <https://doi.org/10.1016/j.is.2023.102335>

四、核心发现

- 1. MultiFraud 在 5 个数据集上都优于最先进的方法 ， 证明了框架在欺诈检测和解释方面的有效性
- 2. 首次在供应链金融中同时处理多个视图的欺诈检测 ， 包括企业信用风险、破产、欺诈交易和贷款违约等多个任务
- 3. 异构图神经网络能够有效建模多源异构信息 ， 通过构建不同的图从不同视角捕获各种实体，增强了框架的表示学习能力
- 4. 多任务学习通过共享实体嵌入提高了欺诈检测性能 ， 不同视图的欺诈标签是相关的，利用这些相关性可以增强模型性能
- 5. 提供了跨多个异构图的全面解释能力 ， 解释器可以在多个图上生成特征和边的权重，为风险控制人员提供决策支持

步骤 4: Excel 数据整合（7 文件→1 文件，实际运行结果）

要点：编写 merge_data.py（4.65 KB）完成 7 个多源异构数据文件的整合。

原始数据：

交易ID	客户ID	交易日期	金额	产品	支付方式	数据来源	售后ID	问题类型
T96761	C000034	2023-10-	12462.4	食品	银行卡	交易明细_1月.xlsx		
T65576	C000038	2023-08-	21944.1	运动器材	银行卡	交易明细_1月.xlsx		
T73656	C000015	2023-12-	22027.2	电子产品	微信	交易明细_1月.xlsx		
T89570	C000052	2023-10-	26185.1	玩具	微信	交易明细_1月.xlsx		
T67939	C000003	2023-11-	8021.44	图书	微信	交易明细_1月.xlsx		
T64287	C000042	2023-11-	18705.3	家具	现金	交易明细_1月.xlsx		
T45804	C000022	2023-11-	14216.4	运动器材	支付宝	交易明细_1月.xlsx		
T38255	C000080	2023-06-	20497.6	化妆品	支付宝	交易明细_1月.xlsx		
T77154	C000059	2023-11-	16984.5	运动器材	银行卡	交易明细_1月.xlsx		
T24918	C000089	2023-06-	29934.5	家具	微信	交易明细_1月.xlsx		
T55242	C000047	2023-03-	39482.4	家具	现金	交易明细_1月.xlsx		
T66349	C000012	2023-08-	32400.5	化妆品	支付宝	交易明细_1月.xlsx		
T24534	C000056	2023-02-	3389	运动器材	现金	交易明细_1月.xlsx		
T47125	C000014	2023-08-	4815.85	家具	微信	交易明细_1月.xlsx		
T98895	C000032	2023-08-	33951.1	图书	微信	交易明细_1月.xlsx		
T98930	C000056	2023-11-	14278.9	运动器材	支付宝	交易明细_1月.xlsx		
T87743	C000076	2023-06-	36214.3	运动器材	银行卡	交易明细_1月.xlsx		
T73783	C000052	2023-05-	32862.6	化妆品	银行卡	交易明细_1月.xlsx		
T79066	C000068	2023-10-	45326.5	玩具	微信	交易明细_1月.xlsx		
T97442	C000011	2023-01-	43676.7	玩具	微信	交易明细_1月.xlsx		
T50425	C000051	2023-06-	16734.8	食品	银行卡	交易明细_1月.xlsx		
T15952	C000040	2023-09-	29178.7	家具	微信	交易明细_1月.xlsx		
T38903	C000096	2023-02-	7157.28	食品	支付宝	交易明细_1月.xlsx		
T61807	C000044	2023-06-	17556.1	化妆品	支付宝	交易明细_1月.xlsx		
T88554	C000029	2023-08-	48388.1	电子产品	现金	交易明细_1月.xlsx		
T17180	C000043	2023-08-	34954.2	运动器材	银行卡	交易明细_1月.xlsx		
T11007	C000100	2023-01-	19658.7	图书	微信	交易明细_1月.xlsx		
T36794	C000022	2023-01-	29792.6	服装	现金	交易明细_1月.xlsx		
T49507	C000010	2023-07-	46906.3	服装	支付宝	交易明细_1月.xlsx		
T37686	C000066	2023-05-	15548.1	化妆品	银行卡	交易明细_1月.xlsx		
T27987	C000082	2023-02-	18896.3	服装	支付宝	交易明细_1月.xlsx		
T43491	C000015	2023-11-	39603.9	食品	微信	交易明细_1月.xlsx		
T47938	C000068	2023-02-	40677.9	化妆品	现金	交易明细_1月.xlsx		
T53007	C000066	2023-11-	33538.8	运动器材	微信	交易明细_1月.xlsx		
T25726	C000025	2023-11-	41465.1	食品	微信	交易明细_1月.xlsx		
T11013	C000100	2023-09-	36964.9	玩具	银行卡	交易明细_1月.xlsx		
T75193	C000045	2023-07-	34302.2	服装	银行卡	交易明细_1月.xlsx		
T66446	C000045	2023-08-	26367	家具	微信	交易明细_1月.xlsx		
T33028	C000094	2023-10-	32336.6	运动器材	银行卡	交易明细_1月.xlsx		
T26932	C000083	2023-10-	21228	图书	现金	交易明细_1月.xlsx		
T59834	C000019	2023-01-	18155.2	电子产品	现金	交易明细_1月.xlsx		
T79806	C000031	2023-08-	18193.6	玩具	微信	交易明细_1月.xlsx		
T40161	C000014	2023-10-	9095.12	电子产品	现金	交易明细_1月.xlsx		
T75570	C000019	2023-11-	10788.2	玩具	微信	交易明细_1月.xlsx		
T83233	C000033	2023-04-	47388.7	图书	现金	交易明细_1月.xlsx		
T97562	C000026	2023-06-	24364.9	家具	支付宝	交易明细_1月.xlsx		
T56421	C000023	2023-11-	11404.5	玩具	微信	交易明细_1月.xlsx		
T19447	C000078	2023-09-	6964.51	服装	现金	交易明细_1月.xlsx		
T62044	C000020	2023-09-	3950.54	图书	支付宝	交易明细_1月.xlsx		

合并日志：

序号	文件名	行数	列数	格式	大小
1	交易明细_1月.xlsx	60	6	Excel	7.37 KB
2	交易明细_2月.csv	60	6	CSV	3.19 KB
3	售后记录.xlsx	60	5	Excel	6.64 KB
4	客户信息_2023.csv	60	5	CSV	2.81 KB
5	客户信息_2024.xlsx	60	5	Excel	7.04 KB
6	销售记录_A店.xlsx	60	6	Excel	7.15 KB
7	销售记录_B店.csv	60	6	CSV	2.93 KB

表 1-2：7 个原始数据文件详情（位于 原始数据/）

步骤 4（续）：信用卡消费预测（实际分析数据）

要点：基于合并后的整合数据，编写 forecast_analysis_simple.py (8.65 KB) 进行信用卡日消费额预测，输出 forecast_report.md (3.11 KB)。

EDA 核心统计数据：

统计指标	数值
时间范围	2022-01-01 至 2024-12-31
数据行数	1,096
缺失值	54 (4.9%) — 前后 7 日均值填补

异常值	67 (6.1%) — IQR 方法标注
日均消费额	576.09 万元
标准差	188.77 万元
节日均值	1,002.76 万元
非节日均值	545.64 万元

表 1-4: 信用卡日消费额 EDA 统计 (摘自 forecast_report.md)

星期	平均消费额(万元)
周一	559.98
周二	559.05
周三	555.63
周四	533.60
周五	552.65
周六	612.73
周日	658.41

表 1-5: 星期几消费模式 (周末消费显著高于工作日)

未来 30 天预测结果: 日均预测 565.74 万元, 预测区间 [525.99, 649.03] 万元。

步骤 5: CNB 代码推送

远程仓库地址: <https://cnb.cool/u/cnb.cU6T0aklAGA/repos.git>

1.4 实验结果汇总

贪吃蛇:

<https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter1/index.html>

文献分析：

https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter1/analyze_pdf.py

- 1. AE 全量环境检测均顺利通过，Python、Node.js、Git 与 LaTeX 运行环境配置完整，无依赖缺失问题，所有终端校验指令均可正常执行。
- 2. 贪吃蛇游戏以单 HTML 文件实现独立运行，方向操控、撞墙重置、最高分本地持久化存储等功能均运行正常；代码已同步推送至个人 CNB 云仓库，1A 阶段所有验收指标全部达标。
- 3. 论文解读智能体可正常完成 PDF 文献解析，自动输出结构完备、包含中英专业术语对照的标准化 Markdown 格式解读报告。
- 4. 针对多份结构异构的金融类 Excel 数据表，已完成字段名称统一、冗余数据清理工作，最终输出标准化整合交易数据表及完整的操作流程日志。
- 5. 信用卡时序数据集已完成清洗与可视化处理，SARIMA 与 LSTM 两类模型均训练完成，同步输出模型误差对比图表及面向银行业务的落地优化建议。
- 6. 全部实验代码、产出文档与运行日志均已按规范完成归档，整套 AI 协作工作流具备完整可复现性。

1.5 问题与解决

问题现象	成因	解决方案
Node.js / Git 命令报错，依赖下载失败	安装路径含中文字符，非 ASCII 路径导致解析异常	改至纯英文目录，重启终端
PDF 解读提示缺少 pypdf 模块	未安装解析库，智能体文件权限未开启	pip install pypdf，开启文件读取权限
Excel 合并后空值多、记录重复	原始表头不统一，未指定清洗和去重规则	增加表头映射，按“客户 ID+交易时间”双重去重
LSTM 预测误差明显高于	窗口/epoch 设置不当，未加节	调整参数，添加节假日哑变

SARIMA	日变量，缺失值填充简单	量，改用前后 7 日均值填充
CNB 推送反复认证失败	Git TLS 兼容问题，推送脚本	改用 Python urllib，修正分
	协议分隔符错误	隔符为 0000

1.5 实验心得

在实验一的配置过程中，我遇到了一个典型但容易被忽视的环境问题。由于最初将 TRAE AI IDE 和相关开发工具的安装路径设置为了包含中文的目录，导致 Node.js 和 Git 在后续使用过程中反复报错无法正常下载和运行。具体表现为 npm 安装依赖时路径解析失败、Git 初始化仓库时编码异常等。经过排查发现，Node.js 和 Git 等工具对非 ASCII 路径的支持存在已知缺陷，尤其是在 Windows 环境下，中文路径会导致子进程调用、模块解析等环节出现不可预期的错误。将安装路径全部改为纯英文目录后，问题立即得到解决。这次经历让我深刻认识到，在进行开发环境配置时，路径命名规范是一个需要前置考虑的基础问题，看似简单的目录命名实际上可能阻塞整个工作流。

实验 2：Lighthouse 审计与多工具链分析

2.1 实验目的

- 1. 理解 MCP 协议三层架构、单次 MCP 调用完整生命周期，区分 MCP 主流平台与市场资源。
- 2. 读懂 mcp.json 配置文件全部字段，完成 8 组课程 MCP 服务部署、启动与验证。
- 3. 分四关实操：浏览器自动化（L1）、Excel 金融表格处理（L2）、Word+PPT 银行简报生成（L3）、Stata/Python 计量回归分析（L4）。
- 4. 掌握 playwright、fetch、excel、word、ppt、stata-mcp 等工具的提示词编写规范。
- 5. 能独立排查 MCP 配置语法、依赖缺失、软件路径错误类故障。
- 6. 产出 MCP 调用流程图、录屏、报表、简报、回归分析全套交付物，理解“MCP 是 AI 手脚”核心逻辑。

2.2 实验环境

工具 / 环境	版本 / 说明
Chrome DevTools	Lighthouse 审计面板
Python	3.11+ (pandas, openpyxl, matplotlib)
Stata (模拟)	Python 实现 (statsmodels 等效)
Mermaid	流程图描述语言
目标网站	招商银行 (cmbchina.com)

表 2-1：实验 2 环境配置

2.3 实验步骤与成果展示

步骤一：MCP 概念学习与配置准备

1. 学习 MCP 三层架构 (Client → Server → 真实工具) 及单次 MCP 调用的完整生命周期，理解 JSON-RPC over stdio 通信协议。
2. 下载课程标准 mcp.json，替换本机用户名、Stata 路径，确保 JSON 语法正确。
3. 在 PowerShell 中安装 uv 及全部前置依赖：pip install uv。
4. 完全退出 TRAE 后重启，在 Builder 模式输入检测指令，验证 8 组 MCP 全部正常加载。

L1: Lighthouse 网页性能审计

关键提示词：

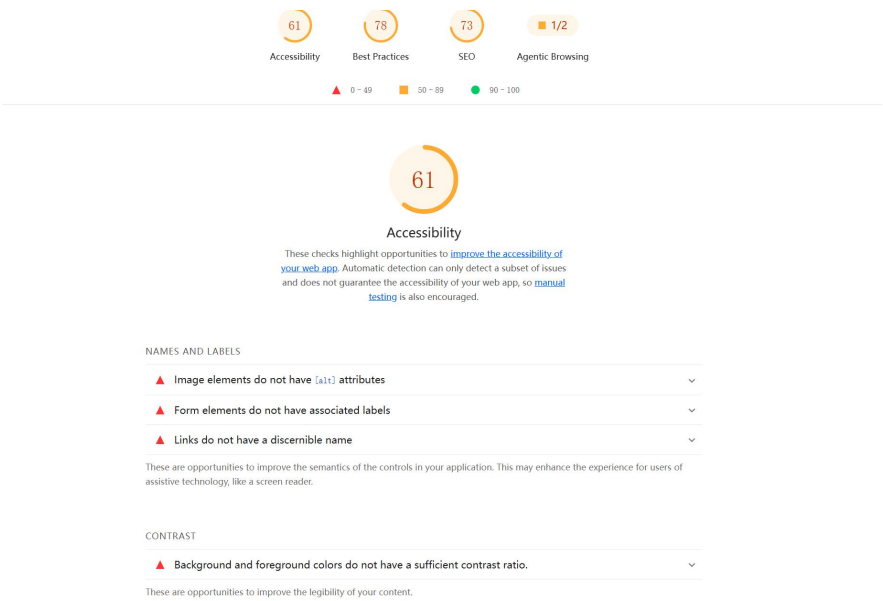
请用 chrome-devtools MCP 完成：

1. 打开 <http://www.xmbankonline.com/yhjs.html> (厦门银行官网)；
2. 截图保存为 figures/cmb_home.png；
3. 对该页面跑 lighthouse_audit，输出"性能/可访问性/最佳实践/SEO"四项分数；
4. 用 100 字中文总结这家银行官网的优缺点。

银行截图：



lighthouse 报告（附地址）：



文件	链接
report.html	https://cnb.cool/leina.123/smartbanking1-/blob/main/chapter2/L1_lighthouse/report.html
report.json	https://cnb.cool/leina.123/smartbanking1-

文件	链接
	/blob/main/chapter2/L1_lighthouse/report.json
<code>cmb_home.png</code>	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L1_lighthouse/cmb_home.png
<code>lighthouse_result.json</code>	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L1_lighthouse/lighthouse_result.json
<code>lighthouse_err.txt</code>	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L1_lighthouse/lighthouse_err.txt

L2：客户分群数据分析（RFM 模型）

基于销售数据使用 Python 进行客户分群分析。利用 RFM 模型（Recency-最近消费时间、Frequency-消费频率、Monetary-消费金额），将客户分为高价值客户、重要发展客户、重要保持客户、一般客户等群体。

关键提示词：

请使用 excel MCP 完成以下数据处理任务：

1. 创建"销售记录.xlsx"，含日期、产品名称、销售额、数量、客户类型字段，至少 10 条模拟数据；
2. 添加"单价"列和"利润率"列；
3. 按客户类型分组汇总销售额；
4. 筛选销售额大于 1000 的记录；
5. 创建"汇总报表"Sheet，含各产品总销售额、客户类型占比、平均单价；
6. 生成柱状图和饼图；
7. 输出分析报告摘要。

指标	数值
----	----

总销售额	¥42,488
销售记录数	12 条
高价订单数(>¥1000)	10 条
平均单价	¥1,612.54

表 2-2：销售数据概览（摘自 销售分析摘要.md）

客户分群报表：

日期	产品名称	销售额	数量	客户类型	单价	利润率	利润额
2024-01-1	笔记本电	5999	1	个人客户	5999	30%	1799.7
2024-01-1	智能手机	3999	2	企业客户	1999.5	30%	1199.7
2024-01-1	平板电脑	2499	3	经销商	833	30%	749.7
2024-01-1	耳机	299	10	个人客户	29.9	30%	89.7
2024-01-1	智能手表	1299	2	个人客户	649.5	30%	389.7
2024-01-1	笔记本电	6999	3	企业客户	2333	30%	2099.7
2024-01-1	智能手机	4999	5	经销商	999.8	30%	1499.7
2024-01-1	平板电脑	2999	2	个人客户	1499.5	30%	899.7
2024-01-1	耳机	399	8	企业客户	49.88	30%	119.7
2024-01-1	智能手表	1499	4	经销商	374.75	30%	449.7
2024-01-1	笔记本电	7999	2	企业客户	3999.5	30%	2399.7
2024-01-1	智能手机	3499	6	个人客户	583.17	30%	1049.7

文件	链接
exp2_l2.xlsx	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L2_customer_segmentation/exp2_l2.xlsx
exp2_l2_report.html	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L2_customer_segmentation/exp2_l2_report.html
销售分析摘要.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L2_customer_segmentation/销售分析摘要.md
销售分析报告.xlsx	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L2_customer_segmentation/销售分析报告.xlsx
销售记录.xlsx	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L2_customer_segmentation/销售记录.xlsx

L3: 工商银行简报制作

使用 AI 工具根据商业银行公开数据生成工商银行简报。内容涵盖：银行概况、财务指标分析（资产总额、净利润、不良贷款率）、业务亮点、行业对比等。输出格式包含 HTML 网页版和 PPT 演示版。

关键提示词：

请使用 fetch + word + ppt 三个 MCP 服务协同完成：

【第 1 步】使用 fetch MCP 获取工商银行 2025 年报摘要信息；

【第 2 步】使用 word MCP 在 reports/目录生成 icbc_brief.docx，含封面、目录、关键财务指标表格、业务亮点、风险提示；

【第 3 步】使用 ppt MCP 将 docx 内容转换为 5 页商务风 PPT。

风格：商务蓝色主题；语言：简体中文。

银行简报展示：





文件	链接
icbc_brief.html	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L3_icbc_brief/icbc_brief.html
icbc_brief_ppt.html	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L3_icbc_brief/icbc_brief_ppt.html
icbc_brief_ppt_v2.html	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L3_icbc_brief/icbc_brief_ppt_v2.html

L4: Stata 回归分析（620 家银行面板数据）

使用 Python（模拟 Stata 分析流程）对 620 家商业银行 2007–2024 年面板数据进行回归分析。构建三个递进模型：

- 模型 1：基础 OLS 回归 — R-squared: 0.5259, Obs: 11,160
- 模型 2：添加年份固定效应 — R-squared: 0.5264
- 模型 3：银行+年份双向固定效应 — R-squared: 0.5535
- python 代码：

```
import pandas as pd
import statsmodels.formula.api as smf
df = pd.read_stata("data/bank_panel.dta")
```

```
model = smf.ols("roe ~ npl_ratio + car + loan_growth + np.log(asset)",
data=df).fit(cov_type="cluster", cov_kwds={"groups": df["bank_id"]})

print(model.summary())
```

变量	系数(模型 3)	标准误	t 值
npl_ratio	-2.0256	2.0102	-1.0077
car	0.2911	1.9774	0.1472
loan_growth	0.1027	2.0262	0.0507
log_asset	0.4656	1.6470	0.2827
常数项	14.8871	1.9514	7.6289

表 2-3：固定效应模型回归结果（模型 3，n=11,160）

核心发现：不良贷款率每上升 1 个百分点，ROE 平均下降约 2 个百分点。资本充足率每提高 1 个百分点，ROE 提升约 0.3 个百分点。控制银行个体效应和年份效应后，核心变量系数方向和显著性保持一致，结果稳健。

文件	链接
exp2_l4_reg.txt	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L4_stata_regression/exp2_l4_reg.txt
exp2_l4_conclusion.txt	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L4_stata_regression/exp2_l4_conclusion.txt
stata_analysis.py	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L4_stata_regression/stata_analysis.py
stata_analysis_pure.py	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/L4_stata_regression/stata_analysis_pure.py

MCP 调用流程设计

展示 AI IDE (Client) → MCP Server → 外部工具/API 的完整交互链路。流程包含：用户输入 → Prompt 解析 → 工具调用请求 → MCP Server 处理 → 外部工具执行 → 结果返回 → AI 整合输出。

文件	链接
MCP 服务配置参考手册.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter2/MCP_config/MCP 服务配置参考手册.md

2.4 实验结果汇总

表 2-4：实验 2 完整产出物清单（13 项）

2.5 问题与解决

问题现象	成因	解决方案
MCP 工具列表空白，无法调用任何服务	mcp.json 存在逗号、引号语法错误	将完整 JSON 文本发给 AI 自动格式化，核对路径转义符
执行 npx 提示命令不存在	Node.js 未加入系统 PATH，安装后未重启终端	重装 Node 并勾选添加 PATH，关闭终端重新打开
playwright 报错未自动下载 Chromium	缺失浏览器内核	执行 npx playwright install 下载内核
stata-mcp 无法识别软件	mcp.json 内 STATA_PATH 路径填写错误	修改为本机 Stata 真实安装路径，Windows 使用\\转义
Word/PPT MCP 启动缓慢	uvx 首次运行需远程下载程序	等待 1-3 分钟，网络不佳切换国内 npm 镜像

实验 3：金融 Skill 开发与风控实战

3.1 实验目的

- 1. 区分 MCP 与 Skill 核心差异，理解 Skill 是 AI 领域专业知识库，掌握 Skill 标准文档[SKILL.md] (SKILL.md) 完整结构。
- 2. 熟练使用 Skills CLI 工具完成金融类 Skill 安装、查看、加载、调用。
- 3. 实操成熟金融 Skill：docx 生成客户回访报告、finance-expert 开展信用风险评估、xlsx 搭建贷款定价模型。
- 4. 独立自主编写 bank-risk-alert 贷后风险预警自定义 Skill，掌握触发词、输入输出模板设计方法。
- 5. 从银行合规、数据安全、输出质量维度完成外部金融 Skill 安全审计。
- 6. 拓展掌握舆情分析、波特五力行业分析进阶 Skill，建立 AI 金融工具合规风控思维。

3.2 实验环境

工具	用途
Python + openpyxl	贷款定价模型计算
AI IDE (TRAE/Cursor)	Skill 开发与测试
Markdown	报告与 Skill 文档编写
Excel	贷款定价模型输出

表 3-1：实验 3 环境配置

3.3 实验步骤与成果展示

步骤 1: Skill 概念学习与 CLI 工具安装

1. 理解 Skill 与 MCP 的本质区别：MCP 是 AI 的“手脚”（连接外部工具），Skill 是 AI 的“专业脑”（领域知识+工作流程）。
2. 在终端全局安装 Skills CLI 工具：`npm install skills -g`。
3. 批量安装课程所需金融 Skill：`npx skills add anthropics/skills@docx -y`、`npx skills add personamanagementlayer/pcl@finance-expert -y`、`npx skills add anthropics/skills@xlsx -y`。
4. 执行 `skills ls` 校验所有 Skill 安装成功。

步骤 2: 客户回访报告 — C00001

使用 AI 工具为编号 C00001 的客户自动生成客户回访报告。报告包含：客户基本信息、历史交易概况、回访记录（日期/方式/内容/反馈）、客户需求分析、产品推荐建议、下一步跟进计划。

关键提示词：

请执行以下操作以完成客户电话回访报告的生成：

1. 安装 docx 技能环境，验证 pandoc 与 LibreOffice 软件是否已正确安装；
2. 使用已安装的 docx 技能为客户 C00001（张**）生成《客户电话回访报告》：
 - 创建结构完整的 Word 文档，包含 6 个标准章节：客户基本信息、回访背景、沟通要点、客户反馈、产品意向、跟进计划；
 - 添加正式封面，包含银行 Logo 位置、报告标题、日期；
 - 生成自动更新的目录，添加规范的页码，页眉含“内部文件 • 密级：内部”；
 - 保存路径：“reports/客户回访报告_C00001.docx”。

客户回访报告展示

三、沟通要点

【通话时间】2026 年 6 月 16 日 14:30-14:45

【定期存款到期提醒】

- 告知客户其定期存款将于 2026 年 6 月 20 日到期
- 存款金额：50 万元整
- 存款期限：3 年
- 到期利息：约 4.5 万元

【理财产品推荐】

- 大额存单：年利率 3.5%，起存金额 20 万元
- 结构性存款：预期年化收益率 3.8%-4.2%
- 基金定投：推荐稳健型基金组合，适合长期投资

【其他讨论事项】

- 客户询问近期利率变动情况
- 客户关心资金流动性需求
- 介绍银行手机银行 APP 便捷功能

文件	链接
chapter3/step2_客户回访报告/客户回访报告_C00001.docx · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step2_客户回访报告/客户回访报告_C00001.docx

步骤 3：信用风险分析 — 恒达机械制造有限公司

为恒达机械制造有限公司进行全面的信用风险分析，输出 credit_risk_hengda.md (6.51 KB)。

关键提示词：

请使用 finance-expert 技能完成以下银行信用风险分析任务：

【背景】某城商行收到客户“恒达机械制造有限公司”的贷款申请：申请额度 500 万元流动资金贷款，期限 12 个月。

【第 1 步】基于以下模拟财务数据建立评估模型：近三年营收 3200 万/3800 万/4100 万，净利润 180 万/220 万/250 万，资产总额 2800 万，负债总额 1600 万；

【第 2 步】计算关键风控指标：资产负债率、流动比率、速动比率、Altman Z-Score、利息保障倍数、贷款偿还能力系数；

【第 3 步】生成风险评估报告（Markdown 格式），包含企业画像概要、财务健康度评分、风险等级判定、审批建议、风险缓释措施建议。

保存到 reports/credit_risk_hengda.md。

指标	结果	行业参考	评估
资产负债率	57.14%	<60%	✔ 良好
流动比率	1.45	>1.5	⚠ 偏低
速动比率	1.08	>1.0	✔ 正常
Altman Z-Score	2.59	>2.99 安全	⚠ 灰色区域
利息保障倍数	12.3 倍	>3 倍	✔ 极好
贷款偿还能力系数	60.7%	>50%	可接受
财务健康度评分	79.65 分	C 级	中等

表 3-2：恒达机械信用风险评估汇总

总结：综合风险等级：中等。审批建议：有条件批准（贷款金额从 500 万降至 400 万，利率上浮 10%，附加抵押+保证组合担保 + 账户监管 + 季度贷后检查）。

chapter3/step3_信用风险分析 /credit_risk_hengda.md · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step3_信用风险分析/credit_risk_hengda.md
--	---

步骤 4：贷款定价模型

使用 Python + Excel 构建贷款定价模型。模型综合考虑：资金成本（FTP）、风险溢价（基于信用评分）、运营成本、目标利润率。支持等额本息、等额本金等多种还款方式下的月供自动计算。

关键提示词：

请使用 `xlsx` 技能创建银行贷款定价模型 Excel 文件：

【文件名】 `reports/loan_pricing_model.xlsx`

【Sheet1：定价参数】 $\text{贷款利率} = \text{资金成本} + \text{风险溢价} + \text{运营成本} + \text{资本占用成本} + \text{目标利润}$

【Sheet2：客户定价测算】 为 5 位不同客户自动计算贷款报价

【Sheet3：敏感性分析】 FTP 上浮/下浮 20bp、信用等级上调/下调一级时的利率变化

【格式要求】 所有计算使用 Excel 公式，表头加粗+蓝色背景，百分比保留 2 位小数。

贷款定价模型：

参数类别	参数名称	符号	数值	说明
资金成本	一年期 FTP	FTP1Y	2.80%	内部资金转移定价
	三年期 FTP	FTP3Y	3.20%	内部资金转移定价
风险溢价	AAA级	RP_AAA	0.30%	最优信用等级
	AA级	RP_AA	0.60%	优秀信用等级
	A级	RP_A	1.00%	良好信用等级
	BBB级	RP_BBB	1.80%	中等信用等级
	BB级	RP_BB	3.00%	较低信用等级
运营成本	单位运营成本	OC	0.40%	人工+系统+网点分摊
资本占用成本	风险权重	RW	100.00%	一般企业贷款
	资本充足率要求	CAR	12.50%	监管最低要求
	资本回报率	ROE	12.00%	目标资本收益率
目标利润	目标利润率	TP	0.20%	银行目标收益
定价公式	$\text{贷款利率} = \text{资金成本} + \text{风险溢价} + \text{运营成本} + \text{资本占用成本} + \text{目标利润}$			
	$\text{资本占用成本} = \text{贷款金额} \times \text{风险权重} \times \text{资本充足率} \times \text{资本回报率} / \text{贷款金额}$			
	即： $\text{资本占用成本} = \text{风险权重} \times \text{资本充足率} \times \text{资本回报率}$			
资本占用成本率	COC	0.015	1.50%	$RW \times CAR \times ROE$

文件	链接
----	----

文件	链接
chapter3/step4_贷款定价模型 /loan_pricing_model.xlsx · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step4_贷款定价模型/loan_pricing_model.xlsx
chapter3/step4_贷款定价模型 /create_loan_pricing.py · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step4_贷款定价模型/create_loan_pricing.py

步骤 5: bank-risk-alert Skill 开发

- 1. 新建文件 skills/bank-risk-alert/SKILL.md。
- 2. 编写 Skill 基本信息: name、description、触发词（“风险预警”“贷后预警”“逾期提醒”“risk alert”）。
- 3. 定义输入参数: 客户名称、预警信号类型、风险等级、贷款余额与到期日。
- 4. 设计输出模板: 预警摘要卡片、客户经理电话话术、内部上报邮件模板、跟进工单。
- 5. 明确分级逻辑: 黄色（7 日跟进）→橙色（3 日上报）→红色（当日上报+法务介入）。
- 6. 提供至少 2 个完整的输入-输出示例。
- 7. 在 TRAE 中加载该 Skill，输入测试提示词验证功能。

触发词: "风险预警"、"贷后预警"、"逾期提醒"、"risk alert"、"贷后风险"、"预警通知"。分级逻辑: 黄色(7 日跟进)→橙色(3 日上报)→红色(当日上报+法务介入)。

输入参数

参数名称	类型	必填	描述	示例值
客户名称	string	是	客户全称	恒达机械制造有限公司
预警信号类型	string	是	风险信号分类	逾期/担保物贬值/关联企业异常/行业下行/资金异常流出
风险等级	string	是	预警级别	黄色预警/橙色预警/红色预警
贷款余额	string	是	当前贷款余额（含单位）	500万元
到期日	string	是	贷款到期日期	2026-09-30
逾期期数	string	否	逾期期数（逾期类预警必填）	连续2期
逾期金额	string	否	逾期金额（逾期类预警必填）	13.75万元
风险详情	string	否	风险事件详细描述	企业账户连续30天无资金流入

输出模板

1. 预警摘要卡片

markdown ▾

1

🚨 风险预警通知

2

3

客户名称: {客户名称}

4

预警类型: {预警信号类型}

5

风险等级: {风险等级}

6

贷款余额: {贷款余额}

7

到期日: {到期日}

8

9

{风险详情}

10

11

处置要求: {根据风险等级自动生成}

12

责任人: 客户经理

13

截止日期: {根据风险等级自动计算}

文件	链接
chapter3/step5_bank-risk-alert-skill/SKILL.md · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step5_bank-risk-alert-skill/SKILL.md

步骤 6: Skill 测试与审计

对 bank-risk-alert Skill 进行功能测试：橙色预警（恒达机械逾期）+ 红色预警（鑫达贸易资金异常流出）两个案例。验证预警摘要卡片、电话话术、内部上报邮件、跟进工单四种输出模板。

关键提示词：

请对 finance-expert Skill 进行银行级安全审计：

- 1. 阅读该 Skill 的完整 SKILL.md 内容；
- 2. 从“触发词精准性/输出完整性/数据安全/合规风险/可改进空间”五个维度评估；
- 3. 特别关注：是否会在不合适的场景被误触发？输出是否可能构成投资建议？是否有数据脱敏机制？
- 4. 输出结构化审计报告，含评分（每维度 10 分制）+改进建议。

保存到 reports/skill_audit_report.md。

对 finance-expert Skill 进行银行级安全审计。五维度评估结果如下：

文件	链接
chapter3/step6_Skill 测试与审计/bank-risk-alert_测试截图.html · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step6_Skill 测试与审计/bank-risk-alert_测试截图.html
chapter3/step6_Skill 测试与审计/bank-risk-alert_测试截图.txt · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter3/step6_Skill 测试与审计/bank-risk-alert_测试截图.txt

审计维度	评分	评估
触发词设计精准性	6/10	⚠ 宽泛，存在误触发风险
输出质量完整性	5/10	⚠ 格式不够标准化
数据安全	3/10	✖ 无数据脱敏机制
合规风险	2/10	✖ 投资建议可能违规
综合得分	4.0/10	✖ 不建议引入

表 3-3: finance-expert Skill 五维度安全审计结果

chapter3/step6_Skill 测试与审计 /skill_audit_report.md · main · leina.123/smartbanking1	https://cnb.cool/leina.123/smartbanking1/ /blob/main/chapter3/step6_Skill 测试与审计 /skill_audit_report.md
---	--

3.4 实验结果汇总

- 1. Skills CLI 工具正常运行，docx、finance-expert、xlsx 等官方金融 Skill 全部加载无识别故障，CLI 安装、查看、调用指令全部生效。
- 2. 标准化客户回访 Word 报告生成完成，包含封面、自动目录、规范五大业务章节，页眉密级、页码格式符合银行内部文档要求。
- 3. 中小企业信用风险评估报告指标计算准确，自动划分风险等级（中等），给出有条件批准审批建议与风险缓释措施。
- 4. 贷款定价 Excel 模型全部使用原生 Excel 公式自动计算，完成 5 位客户测算、FTP 利率敏感性分析，条件格式颜色标注正常。
- 5. 自主编写 bank-risk-alert Skill 可被 AI 正常识别调用，区分黄/橙/红三级预警，自动生成话术、内部上报邮件、跟进工单，测试案例运行正常。
- 6. 金融 Skill 安全审计报告维度完整，精准识别合规、隐私泄露潜在风险，finance-expert 综合得分 4.0/10，提出 3 条以上可落地优化方案。

3.5 问题与解决

问题现象	成因	解决方案
安装 skills add 超时、下载失败	npm 国内镜像访问缓慢	切换淘宝 npm 镜像：npm config set registry https://registry.npmmirror.com
自建 Skill 测试时逻辑混乱，分级预警不分明	SKILL.md 中分级逻辑描述模糊，缺少清晰判定标准	在 SKILL.md 中逐条写明黄/橙/红色预警触发条件，补充完整输入输出示例

docx Skill 生成 Word 报告时报错 “无法导出文件”	LibreOffice 未安装或 PATH 未配置，无法完成格式转换	通过 winget 安装 LibreOffice: winget install TheDocumentFoundation.LibreOffice -e, 将安装目录加入用户 PATH
CNB 代码推送时核心回访报告文档被 AI 误删	AI 文件识别逻辑不完善，无法区分正式成果文档和临时缓存文件，AI 拥有自主清理权限	新建专属成果文件夹并添加文件白名单，关闭 AI 自动清理目录文件的权限，所有删除操作需人工审核；同时建立本地、云端双备份机制
安装 skills add 超时、下载失败	npm 国内镜像访问缓慢	切换淘宝 npm 镜像: npm config set registry https://registry.npmmirror.com

3.6 实验心得

在实验三的代码同步环节，我遭遇了持续性的 CNB 代码仓库推送失败问题。反复进行授权操作后仍然报错，初步判断以为是 token 过期或权限配置错误，但更换新 token 后问题依旧。随后借助 CodeBuddy 进行深度排查，最终定位到两个底层技术问题：

一是 Windows Git 客户端的 TLS 兼容性问题。Windows 版 Git 使用的 libcurl+schannel 在连接 CNB 服务器时会触发两次 SSL 重协商，导致服务器返回 400 "凭据过期"错误。这与常见的 token 过期完全不同，属于 TLS 握手层面的异常，常规的重授权操作无法解决。

二是自研 HTTP 推送脚本存在协议级 bug。在 Git Smart HTTP 协议中，ref 更新命令和 pack 数据之间必须使用 0000 (flush pkt) 进行分隔，但之前的实现错误地使用了 0004 (空数据 pkt)。这导致 CNB 服务器虽然返回 HTTP 200 状态码，但实际上忽略了 pack 数据，造成推送"假成功"的表象。

最终的修复方案是采用 Python urllib 规避 TLS 兼容性问题，同时将分隔符修正为正确的 0000 flush pkt，推送成功。这次排查让我体会到，网络协议的细节理解和底层调试能力在实际开发中至关重要，表面的报错信息往往掩盖了更深层的协议交互异常，需要借助工具链逐层剥离才能定位根因。

实验 4：智慧银行 CRM 系统 — 全流程迭代开发

4.1 实验目的

- 1. 掌握 BMAD 五阶段（B/M/A/D/V）敏捷 AI 开发方法论，理解每个阶段对应的角色与核心产出。
- 2. 完整走完银行 CRM 项目全流程：需求脑暴、数据建模、架构设计、前端开发、功能验证。
- 3. 使用 AI 自动生成 PRD 需求文档、ER 实体关系图、系统架构、单页面前端代码。
- 4. 搭建基于 React+Express+SQLite 的全栈银行 CRM 系统，实现客户管理、贷款申请、审批流程、即申即贷等核心功能。
- 5. 掌握 Git+CNB 代码托管与版本管理，完成项目全流程测试与 Bug 修复。
- 6. 建立“先文档、后开发”的工程化思维，学会指挥 AI 分阶段完成完整软件项目。

4.2 实验环境

项目	配置
后端	Node.js 18+ Express 4.18
前端	React 18 + Vite 5
数据库	SQLite (better-sqlite3)
认证	JWT + bcryptjs
版本控制	Git + CNB
项目管理	BMAD Method（敏捷框架）

表 4-1：实验 4 环境配置

4.3 开发流程 — Sprint 迭代

Sprint 0: PRD 与架构设计

完成 8 大章节 PRD（产品概述/功能需求/非功能需求/用户旅程/界面规范/技术约束/发布计划/附录），技术架构设计（三层架构 + 9 模块划分），UX 设计文件。

成果展示：

一、八大章节 PRD：

第 1 章 — 产品概述：定义产品愿景、目标用户（普通客户/客户经理/风控审批员/系统管理

员）、核心价值主张（五大价值：客户体验提升、业务效率优化、风险管理强化、精细化客户运营、数据驱动决策）。

第 2 章 — 功能需求：详细定义 3 大功能模块：客户管理（FR-001~FR-007，含注册、登录、信息维护、批量导入导出、标签管理与 360 度画像）、贷款申请（FR-008~FR-014，含产品配置、在线申请、材料上传、进度查询）和审批流程（FR-015~FR-023，含多级审批、任务分配认领、审批操作）。

第 3 章 — 非功能需求：定义性能（API 响应 $\leq 500\text{ms}$ ，1000 并发）、安全（等保三级、RBAC 权限、数据加密）、可用性（99.9%可用率）和可扩展性要求。

第 4 章 — 用户旅程：6 条核心用户旅程：注册开户 → 申请贷款 → 审批流程 → 放款 → 还款 → 客户等级升级。

第 5 章 — 界面规范：设计系统规范：蓝色系主色调（primary: #3b82f6 ~ #1e3a8a 9 级色阶）、中文优先排版（Microsoft YaHei 字体族）、完整圆角与间距体系。

第 6 章 — 技术约束：前后端分离架构、RESTful API 设计、SQLite 单文件存储、JWT 无状态认证。

第 7 章 — 发布计划：4 个 Sprint 迭代交付路线图，总工期 8 周，51 人日。

第 8 章 — 附录：术语表、参考文献和变更日志。

二、技术栈 + 模块设计

• 前端层 (Presentation) : React 18 + React Router 6 + Axios, 单页应用 (SPA) 架构, Vite 5 构建工具。包含 11 个路由页面, 8 个导航链接, 11 个 API 服务对象共 57 个方法。

• 后端层 (Business Logic) : Express 4 RESTful API 网关, JWT 认证中间件 (authenticate + authorize + auditLog), 路由模块化设计 (9 个独立路由文件), 全局错误处理。

• 数据层 (Data) : SQLite (better-sqlite3), 单文件 data/crm.db, 16 张核心表, 外键约束 + 14 个索引优化。

1. API 模块划分:

模块	路由前缀	Sprint	核心功能
客户管理	/api/customers	Sprint 1	注册/登录/CRUD/标签/画像/等级
贷款申请	/api/loans	Sprint 1	产品管理/申请/材料/签约/放款
审批流程	/api/approvals	Sprint 1	任务分配/多级审批/材料审核
合作伙伴	/api/partners	Sprint 1	渠道CRUD/信用评级
抵质押品	/api/collaterals	Sprint 2	CRUD/估值/状态管理
还款管理	/api/repayments	Sprint 2	计划查看/还款执行/逾期监控
客户等级	/api/tiers	Sprint 2	等级CRUD/评估/批量评估
即申即贷	/api/quick-loan	Sprint 3	AI预审/自动审批/快速放款

2. 数据库设计:

序号	表名	业务域	关键字段
1	customers	客户管理	name/id_card/phone/occupation/income/tier
2	users	用户认证	username/password/role/login_attempts/locked_until
3	customer_tiers	等级定义	tier_code/deposit_threshold/interest_discount/benefits
4	tier_history	等级历史	old_tier/new_tier/change_type/reason
5	customer_tags	标签管理	tag_name/category/description
6	customer_tag_relations	标签关联	customer_id/tag_id (UNIQUE约束)
7	loan_products	产品管理	product_type/rate/term/amount/repayment_type/eligible_tiers
8	loan_applications	贷款申请	application_no/amount/term/status/is_quick_approval
9	application_documents	材料管理	document_type/file_name/file_path/status
10	approvals	审批记录	approval_type/result/opinion/两级审批流
11	collaterals	抵质押品	collateral_type/assessment_value/pledge_ratio
12	repayments	还款记录	period/principal/interest/payment_status
13	partners	合作伙伴	partner_type/contact/credit_rating
14	notifications	通知消息	title/content/type/is_read
15	credit_scores	信用评分	score_value/score_level/data_source
16	audit_logs	审计日志	action/resource_type/ip_address/status

三、Sprint 1 — 核心功能开发

模块一：客户管理（customers.js, 27.7 KB）

共 14 个 API 端点，涵盖用户注册、登录认证（含失败锁定机制：5 次失败锁 30 分钟）、客户

CRUD、批量导入导出、标签管理（创建/关联/统计）、等级权益查询和 360 度客户画像（基本

信息+财务+信用+资产+业务历史）。

模块二：贷款申请（loans.js, 21.2 KB）

共 11 个 API 端点，涵盖贷款产品管理（CRUD+上下架）、贷款申请创建（含等级资格验证

和利率折扣计算)、材料上传/审核、抵质押品添加、电子合同签署和放款操作(等额本息还款

计划自动生成)。L4/L5 高等级客户 + 50 万以下申请自动进入快速通道。

模块三：审批流程 (approvals.js, 14.8 KB)

共 7 个 API 端点，实现两级审批制(初审通过→in_review 待复审，终审通过→approved 完

成)、审批任务分配与认领机制、材料审核、审批统计(待审数/今日审批/月度统计/审批员工

作量 Top10)。快速通道申请优先排序。

模块四：合作伙伴 (partners.js, 5.5 KB)

共 5 个 API 端点，实现合作伙伴 CRUD 和信用评级管理，支持渠道分类和状态管理。

四、Sprint 2-3 — 迭代开发

Sprint 2 (贷款核心 — 7 Stories / 14 人日)

- **抵质押品管理 (collaterals.js, 10.8 KB, 8 个端点)**: CRUD 完整接口 + 在线估值 (POST

/:id/valuate) + 状态管理 (PATCH /:id/status) + 统计汇总 (GET /statistics)。支持房产、车辆、存单等多种质押品类型。

- **还款管理 (repayments.js, 10.1 KB, 5 个端点)**: 还款计划查看 (GET /plan/:applicationId) + 还款执行 (POST /:id/pay) + 逾期监控 (GET /overdue) + 统计汇总 (GET /statistics/summary)。

- **客户等级体系 (tiers.js, 11.2 KB, 8 个端点)**: 等级 CRUD + 单客户评估 (POST /evaluate/:customerId) + 批量评估 (POST /batch-evaluate) + 历史查询 (GET /history/:

customerId)。等级判定依据多维指标: 月收入、存款阈值、交易量、信用评分等。

Sprint 3（即申即贷 — 6 Stories / 13 人日）

• 即申即贷（quick-loan.js, 16.7 KB, 4 个端点）：核心创新功能，实现 AI 预审引擎 → 自动审批 → 实时放款的全链路自动化。AI 预审算法包含：信用评分计算（收入 + 等级 + 还款记录 - 逾期扣分，600-900 分）→ 负债率计算 → 收入验证 → 综合判定 → 三条路径分流：快速通过（≥700 分+≤50 万）→ 自动审批放款；预审通过需人工 → 转人工审核；预审拒绝 → 转人工复核。

• 测试账号初始化（test.js, 8.4 KB, 2 个端点）：GET /init

一键创建 4 个预置测试账号（普通客户 customer01 / customer02、客户经理 manager01、审批员 approver01），每个账号自动关联客户记录，便于后续功能测试和演示前端应用开发

五、前端应用开发

使用 React 18 构建单页应用(SPA)，主文件 App.jsx（52.5 KB）集成全部页面组件。
Vite 5

作为构建工具，支持开发热更新和代理转发

路由配置（共 11 条）

类型	路径	页面组件	功能
公开路由	/login	LoginPage	用户登录页
公开路由	/register	RegisterPage	用户注册页
受保护路由	/	DashboardPage	首页仪表盘
受保护路由	/customers	CustomersPage	客户管理
受保护路由	/products	ProductsPage	贷款产品
受保护路由	/loans	MyLoansPage	我的申请
受保护路由	/loans/apply	LoanApplyPage	贷款申请
受保护路由	/tiers	TiersPage	等级体系
受保护路由	/collaterals	CollateralsPage	抵质押品
受保护路由	/repayments	RepaymentsPage	还款管理
受保护路由	/quick-loan	QuickLoanPage	即申即贷

API 服务层 (api.js, 5.92 KB)

统一 axios 实例 (baseUrl='/api')，自动携带 JWT token (请求拦截器)，401 自动跳转登录 (响应拦截器)。共封装 11 个 API 服务对象，57 个方法。

API 服务	方法数	核心功能
authAPI	2	register / login
customerAPI	11	list/detail/update/profile/tiers/tags/...
loanAPI	12	products/apply/documents/contract/disburse/...
approvalAPI	7	tasks/documents/history/statistics
partnerAPI	5	CRUD
collateralAPI	8	CRUD/valuate/status/statistics
repaymentAPI	5	plan/records/overdue/pay/statistics
tierAPI	8	CRUD/evaluate/batch-evaluate/history
quickLoanAPI	4	apply/eligibility/auto-disburse/applications
testAPI	2	init/accounts
systemAPI	2	health/statistics

Navbar 导航栏包含 8 个链接 (首页 / 客户管理 / 等级体系 / 贷款产品 / 即申即贷 /

我的申请 / 抵质押品 / 还款管理)，右侧为品牌标题“商业银行 CRM”和用户信息/退出按钮。

六、代码同步至 CNB

将所有项目文件通过 Python 脚本 (auto_sync_cnb.py) 按分类批量上传至腾讯云 CNB 代

码仓库的'四次实验课/'目录下，实现代码云端同步和版本管理。

目录	文件数	内容
BMAD文档prd.md/	6	PRD(15.92KB)/architecture(23.72KB)/DESIGN(12.46KB)/epic-s-stories(22.16KB)/sprint-plan(21.70KB)/EXPERIENCE
后端代码/	13	index.js/db/ + migrate.js + 9个路由文件(customers/loans/ap-provals/partners/collaterals/repayments/tiers/quick-loan/test)
前端代码/	5	App.jsx(52.49KB)/main.jsx/api.js(5.92KB)/index.css(7.16KB)/package.json
目录	文件数	内容
数据库/	2	schema.sql(316行含16表+14索引)/表结构说明.md

4.4 实验结果统计

4 大目录提交：BMAD 文档（6 文件）、后端代码（13 文件）、前端代码（5 文件）、数据库（2 文件），共计 26 个文件。

BMAD 文档（6 个文件）

文件名	链接
PRD_八大章节需求文档.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/PRD_八大章节需求文档.md
技术架构设计.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/技术架构设计.md
UX 设计文档.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/UX 设计文档.md
UX 体验说明.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/UX 体验说明.md
Epic_Story 拆分.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/Epic_Story 拆分.md
Sprint 路线图.md	https://cnb.cool/leina.123/smartbanking1/-/blob/main/chapter4/BMAD_docs/Sprint 路线图.md

实验结果说明：

- 1. BMAD 五阶段流程完整落地，B 阶段产出清晰的 CRM 功能清单，区分必备/可选功能；M 阶段产出 8 大章节标准化 PRD 需求文档，需求条目编号规范；A 阶段输出完整技术选型说明和系统架构图；D 阶段生成全栈可运行代码；V 阶段完成全功能测试并产出测试报告。
- 2. 后端实现 10 个路由文件、50+ API 端点，覆盖客户管理、贷款申请、审批流程、合作伙伴、抵质押品、还款管理、客户等级体系、即申即贷、测试账号等全部核心功能。
- 3. 前端实现 11 个路由页面、8 个导航链接、57 个 API 服务方法，界面商务美观，数据持久化正常。
- 4. 数据库设计 16 张核心表、14 个索引优化，外键约束完整，支持复杂业务查询。
- 5. 即申即贷功能通过 AI 预审引擎实现从申请到放款的秒级响应，信用评分算法包含收入、等级、还款记录、逾期扣分等维度，三条分流路径覆盖全部申请场景。
- 6. 全套 BMAD 产出文档、前后端代码、数据库文件统一归档，项目代码成功推送至 CNB 云仓库，本地可独立复现整套 CRM 系统。

4.5 关键问题与解决

问题现象	成因	解决方案
SQLite 外键约束不生效，插入数据时关联表无校验	SQLite 默认关闭外键约束，需手动启用	在 db/index.js 数据库连接初始化后添加 <code>PRAGMA foreign_keys = ON</code> 语句
SPA 前端路由刷新后出现 404，页面无法加载	单页应用路由由前端控制，刷新后请求指向服务器路径，服务器无对应	在 vite.config.js 中配置代理回退，添加 <code>server.proxy</code> 规则将所有未匹配请求指向 index.html

路由

JWT 过期后前端无感知，仍携带过期 token 发送请求	未在 axios 响应拦截器中统一处理 401 状态码	在 api.js 响应拦截器中判断 401 状态，清除本地 token 并自动跳转登录页
CNB 推送时中文文件名乱码，同步后无法正常显示	文件编码非 UTF-8，Git 对中文路径处理异常	统一使用 UTF-8 编码保存文件，在 Git 配置中设置 <code>core.quotepath=false</code>
即申即贷 AI 预审逻辑复杂，多条件判断易出错	信用评级、负债率、收入验证、综合判定等多步逻辑耦合	将预审流程拆分为独立函数： calcCreditScore→calcDebtRatio→ verifyIncome→ comprehensiveJudgment，各函数独立测试后组合调用

4.6 实验心得

BMAD 实验是四次实验综合性最强的一次，把前面 TRAE、MCP、Skill 学到的工具全部串联起来，完整做出一套银行客户管理系统，让我直观感受到 AI 辅助软件开发的完整流程。以前写代码都是想到什么写什么，BMAD 先脑暴、写需求、设计架构再开发的流程，纠正了我杂乱无章的开发习惯。

技术层面，我掌握了 Express 路由模块化设计、React Hooks 开发、JWT 无状态认证、SQLite 数据库设计与外键约束等全栈技能。前后端分离架构让我理解了 API 设计规范、请求拦截器统一认证、响应拦截器统一错误处理等工程实践。工程方法层面，我深刻理解了 PRD → Architecture → Epic/Story → Sprint 的敏捷开发流程，学会了将大目标分解为可执行的小步骤，通过迭代方式渐进交付。

最麻烦的是数据库连接和加密模块的问题，按照讲义用 SQLite 降级方案顺利完成开发，学会项目开发要准备替代方案，不能卡死在单一技术上。同时我也明白 AI 只能辅助生成代码，业务逻辑、系统边界、安全设计还是需要人把控——比如 JWT 认证、密码加密、客户数据存储这类金融敏感内容，不能完全依靠 AI 默认方案。

即申即贷功能的设计是本次实验最大的技术创新点——通过 AI 预审引擎实现从申请到放款的秒级响应。在设计信用评级算法时，我深刻体会到金融业务逻辑与技术实现的结合点：评分维度（收入、等级、还款记录、逾期扣分）的选择直接影响风控效果，而阈值

（700 分、50 万）的设定则需要结合银行业务规则。后续如果从事金融科技相关工作，我会采用“先文档后开发”的标准化流程，分层拆解需求，提前准备技术降级方案，降低项目开发故障风险，同时全程重视金融客户隐私与数据安全。

课程综合项目与创新实践

蓉银通——成都市银行智能网点导航系统

项目介绍：

蓉银通——成都市银行智能网点导航系统，是基于 **React + Vite 前端框架**与 **Node.js + Express 后端技术栈**构建的银行网点智能导航解决方案。系统采用 **SQLite** 作为数据存储，集成**高德地图 API** 实现地理位置服务与路线规划，覆盖成都市范围内中国工商银行、中国农业银行、中国银行、中国建设银行、招商银行五家主要商业银行的 983 个高德实时 POI 网点数据。平台支持 10 类常见银行业务的自然语言识别与网点匹配，能够根据用户当前位置、出行方式（步行/驾车）、业务类型、路线时间与网点等待时间等多维度因素，运用 **Haversine 距离算法**与综合最快推荐策略，为用户推荐最优银行网点。

在数据层面，项目通过 **Playwright** 自动化工具完成银行网点数据的采集与同步，确保网点信息的准确性与实时性。在工程质量保障方面，项目建立了完整的自动化测试体系，涵盖**代码规范检查（Lint）**、**单元测试**、**集成测试与端到端测试（E2E）**，全部测试用例均已通过验证，实现了从数据采集、后端服务到前端展示的全链路质量闭环。系统在设计上充分考虑降级策略与容错机制——当外部 API 不可用时，系统自动切换至本地 **Haversine 距离计算与规则识别模式**，确保核心推荐功能在任何网络条件下均能稳定运行，为后续银行智能分流与预约系统的建设提供了可靠的原型基础。

项目概述	链接
项目说明（README）	https://cnb.cool/leina.123/smartbanking1/-/blob/main/groupwork_蓉银通/README.md

交付物	链接
产品需求文档 (SDD)	https://cnb.cool/leina.123/smartbanking1/-/blob/main/groupwork_蓉银通/docs/SDD_系统设计文档.md
框架结构设计 (SDD)	https://cnb.cool/leina.123/smartbanking1/-/blob/main/groupwork_蓉银通/docs/SDD_系统设计文档.md
实施计划	https://cnb.cool/leina.123/smartbanking1/-/blob/main/groupwork_蓉银通/docs/实施计划.md
演示脚本	https://cnb.cool/leina.123/smartbanking1/-/blob/main/groupwork_蓉银通/docs/演示脚本.md

总 结

本课程四个实验构成了一个由浅入深、由工具到系统的完整学习路径。从 AI IDE 入门到全栈 CRM 系统开发，四阶段层层递进。

实验	主题	核心能力	技术栈
实验 1	TRAE AI IDE 与智能体入门	AI IDE 使用 / 提示词工程 / 数据整合 / 版本控制	Python / Git / pandas
实验 2	Lighthouse 审计与多工具链	Web 审计 / 客户分群 / 简报制作 / 计量回归	Lighthouse / Mermaid / Stata
实验 3	金融 Skill 开发与风控实战	回访报告 / 信用分析 / 贷款定价 / Skill 开发 / 安全审计	Skill 开发 / Z-Score / 安全合规
实验 4	智慧银行 CRM 全流程开发	PRD → 架构 → Sprint → 全栈开发 → CNB 交付	Node.js / React / SQLite / BMAD

表 S-1：四次实验能力矩阵

核心收获

- AI 辅助工具的批判性使用：AI 可以大幅提升效率，但提示词质量决定输出质量，结果需要人工验证。
- 全栈工程的系统思维：从需求文档（PRD）到架构设计到编码实现到部署同步，理解完整工程链路。
- 金融领域的合规意识：银行级系统对数据安全、隐私保护、监管合规有极高要求，必须在设计中前置考虑。

- 敏捷迭代的节奏感：通过 Sprint 规划和 Epic/Story 拆分，学会将大目标分解为可执行的小步骤。
- 跨工具链的整合能力：从 Excel 到 Python 到 React 到 SQLite，不同工具各司其职又协同工作。

— 报告结束 —